

Rapport de Projet de PFA

Lucas Rooy, Sacha Le Hir Mazé

Présentation du Jeu et tutoriel

Il s'agit d'un jeu de puzzle, basé sur des chapeaux que l'on peut prendre en main et lancer, en pouvant se servir de leurs effets.

Il existe trois chapeaux différents dans le jeu :

- Le fez lorsque lancé contre une surface
- Le haut-de-forme que l'on peut utiliser comme trampoline
- Le béret qui se comporte comme une plateforme volante

Le but du jeu est de rejoindre un porte-manteau en utilisant les chapeaux. Le porte-manteau constitue alors une porte permettant de passer au niveau suivant, ou de finir le jeu.

Sur le build `SDL`, le jeu utilise les touches suivantes :

- `q` pour se déplacer à gauche
- `d` pour se déplacer à droite
- `[SPACE]` pour sauter
- `e` pour interagir
 - Lorsqu'on porte un chapeau, cela permet de le lancer
 - Lorsqu'on est à proximité d'un chapeau, cela permet de le porter

Pour le lancer, il suffit de faire `make run_sdl`, car nous nous sommes concentrés sur la version SDL du jeu. Il est possible d'utiliser Nix pour obtenir un environnement de développement, en faisant `nix develop` à la racine.

Nous possédons actuellement quatre niveaux.

- Un premier niveau permet simplement de se familiariser avec les contrôles
- Un second niveau permet de découvrir le fonctionnement du haut de forme
- Un troisième niveau permet de découvrir le fonctionnement du fez
- Un quatrième niveau permet de découvrir le fonctionnement du béret
- Un cinquième niveau montre les possibilités de puzzle du jeu
- Un sixième niveau pour dire au revoir!

Le fez

Le fez explose les blocs à une distance maximale de `30` pixels autour de lui lorsque projeté contre une surface à une vitesse de norme minimale `0.2`, qui est grossièrement la vitesse de lancer par défaut.



Le béret

C'est la seule entité (en dehors des blocs) immune à la gravité et aux frottements du sol. Ainsi, une fois lancé (ou poussé), il devient une plateforme mouvante permettant de traverser des puits.



Le haut de forme

Le haut de forme se comporte comme un trampoline : sauter dessus augmente la vitesse du saut de **0.75** pixels/seconde à **1.2** pixels/seconde et permet d'accéder à des zones inaccessibles. lateforme permettant de traverser des puits.



Le porte-chapeau

Il s'agit de l'objectif du jeu ! A l'aide des chapeaux, il faut rejoindre cet objet présent sur la carte et appuyer sur la touche **e** à proximité du porte-chapeau afin de passer au niveau suivant.



Présentation du Code

Préambule : Modification de Gfx et Ecs

La boucle est appelée avec un couple (`ticks`, `dt`) (le nombre de ticks (unité pas importante) depuis le début du jeu, le nombre de ticks depuis la dernière actualisation). C'est plus pratique.

Sur suggestion de Adrien, et avec son patch, nous avons modifié `Ecs` qui ne supprimait pas correctement les entités.

Composants et entités

Nous avons définis des composants permettant de créer les types d'entités :

- `drawable`, pouvant être dessiné
- `animated`, pouvant être dessiné et animé. Dans ce cas, le champ `tick_speed` correspond au nombre de ticks à attendre avant de changer de texture. Le champ `animation_callback` est un thunk, appelé lorsque l'animation a effectué un tour (et est sur le point de revenir à 0).
- `collidable`, pouvant être sujet aux collisions
- `physics`, pouvant être déplacé et accéléré par les systèmes `Move` et `Physics`.
- `explodable`, pouvant être explosé. Cet entity type est aussi utilisé par l'entité `explosion`, qui porte alors un tag particulier indiquant la force de l'explosion.

Voici un inventaire des entités que nous avons ajoutées :

- `player` est un joueur
- `hat` est une entité représentant un chapeau, qu'il soit porté ou non
 - Si le chapeau est porté, alors il est désenregistré de tous les systèmes et ajouté dans le tag du joueur (tag `Player of hat`)
 - Si le chapeau est présent dans le monde, alors il est enregistré dans les systèmes appropriés et présent dans la liste `wild_hats` accessible via `Global`.
- `block` est un rectangle ayant une texture, une position, une taille, et la possibilité d'être cassé (par une explosion). Il s'agit d'une entité très présente dans le jeu car elle est utilisée pour construire les parois du puzzle.
- `explosion` est une explosion. Par simplicité, elle est de sur-type `explodable`. Le centre de l'explosion et le centre de sa box (ajouté à sa position), et la force de l'explosion est alors la valeur de `force` dans son tag qui doit être `TExplosion force`. Si le tag est différent, l'explosion est considéré comme un `explodable` classique (pouvant être explosé).

Systèmes

Les systèmes `Physics`, `Move` et `Collision` sont exactement ceux faits durant les TPs.

Nous possédons aussi un système `Animation`, qui se charge de modifier la texture d'un objet lorsque sa `tick_speed` est passée, afin de l'animer.

Le système **Camera** est intéressant, car il permet de définir une caméra ayant une position et un zoom; cela est utile pour l'effet de tremblement lors d'une explosion

Enfin, le système **Explosion_sys** (nommé de cette façon pour ne pas rentrer en collision avec le module de l'entité **Explosion**) régit les explosions.

Architecture générale

Le code est structuré autour du module **Level**, qui permet de charger et décharger des niveaux, du module **Game**, qui constitue l'entrypoint du jeu et se charge de l'initialisation et du chargement des textures, et du module **Inputs**, qui permet l'interaction avec le joueur.

Architecture des chapeaux

Les chapeaux ont des fonctionnalités complexes, différentes pour chacun d'entre eux :

Le Fez

On utilise le resolver des collisions pour détecter lorsqu'il faut faire une explosion. Dès que c'est le cas, on supprime le fez et on crée une entité explosion à sa position.

Le Haut de Forme

Celui-ci possède son fonctionnement dans le resolver des collisions du player, car c'est dans ce resolver que l'on décide le type de surface sur laquelle le player se trouve.

Si c'est un Haut de Forme, alors on saura au moment de sauter qu'il faut sauter avec une vitesse plus élevée

Le Béret

Celui-ci est plus complexe. Dans son resolver, on remet sa hauteur et sa vitesse aux bonnes v

Architecture des chapeaux

En général, les chapeaux peuvent être attrapés (nearest neighbour) par le joueur et portés, après quoi on désactive tous leurs systèmes, et leur affichage est réalisé en même temps que celui du joueur.

Lorsqu'il est lancé, le comportement peut être un peu différent entre certains chapeaux, mais on le réenregistre simplement dans leur systèmes.

Les chapeaux ont des fonctionnalités complexes, différentes pour chacun d'entre eux :

Le Fez

On utilise le resolver des collisions pour détecter lorsqu'il faut faire une explosion. Dès que c'est le cas, on supprime le fez et on crée une entité explosion à sa position.

Le Haut de Forme

Celui-ci possède son fonctionnement dans le resolver des collisions du player, car c'est dans ce resolver que l'on décide le type de surface sur laquelle le player se trouve.

Si c'est un Haut de Forme, alors on saura au moment de sauter qu'il faut sauter avec une vitesse plus élevée

Le Béret

Celui-ci est plus complexe. Dans son resolver, on remet sa hauteur et sa vitesse aux bonnes valeurs, et dans le système des collisions, on met l'élasticité entre le Béret et les murs à 1, afin qu'il ne soit pas ralenti de toute façon (inutile après coup, il s'agit d'un artefact).

Présentation de deux points techniques

Interpolation du temps pour les collisions

Pour faire la gestion des collisions, on utilisait au départ ce code simpliste :

game.ml (avant)

```
let update (ticks, dt) =  
  let _ = Input.handle_input (ticks, dt) in  
  Physics_system.update (ticks, dt);  
  Move_system.update (ticks, dt);  
  Collision_system.update (ticks, dt);  
  Camera_system.update (ticks, dt);  
  Animation_system.update (ticks, dt);;
```

Or, si pour une raison ou une autre, le `dt` entre deux appels de `update` est trop grand (c'est-à-dire, suffisamment grand pour qu'un objet avec une vitesse suffisamment élevée traverse un obstacle sans déclencher de collisions), certaines collisions sont ignorées.

Cette situation se produit dans beaucoup de cas :

- Si, peut-être, le système d'exploitation décide de laisser dormir notre jeu pendant trop longtemps. Dans ce cas, on pourrait utiliser `nice` pour sans sortir, à la rigueur, mais ce n'est pas optimal
- Si les calculs et l'affichage de la frame précédente ont été trop longs; cela se produit couramment, en particulier lorsque nous avons beaucoup d'entités présentes.

Afin de régler le problème, on effectue une interpolation du temps à chaque update.

Supposons qu'entre deux appels à `update` se soient passées `dt=200ms`. On effectuera alors les étapes de calcul (move, physics, etc) 200 fois, avec un pas de temps de `1ms` à chaque fois, afin de ne pas perdre en précision.

Ainsi, chaque grand pas de temps est décomposé en petits pas de temps (donc en plus petits mouvements) que l'on peut analyser finement; on a moins de chances de perdre des collisions!

Bien sûr, cette approche possède des inconvénients, dont un particulièrement visible : son impact sur les performances non négligeable, pour y pallier, nous divisons par `5` le nombre de pas.

game.ml (après)

```
let last_ticks = ref 0.  
  
let update (ticks, dt) =  
  if !last_ticks +. 2000. < ticks  
  then (  
    Printf.printf "Current dt = %f\n" dt;  
    last_ticks := ticks);
```

```

for i = 0 to int_of_float (ceil (dt /. 5.)) do
  let dt = ticks, dt /. ceil (dt /. 5.) in
  let () = Input.handle_input dt in
  Physics_system.update dt;
  Move_system.update dt;
  Collision_system.update dt
done;
Camera_system.update (ticks, dt);
Animation_system.update (ticks, dt);
None
;;

```

Méthode de stockage des niveaux

Pour stocker les niveaux, on a décidé d'utiliser les types suivants :

```

type layer =
{ contents : string;
  offsets : (selector, Vector.t ) Hashtbl.t;
  width : int;
  stride : Rect.t;
}

type level =
{ layers : layer list;
  camera : float * Vector.t;
}

```

Les niveaux sont des quadrillages de blocks de taille élémentaire donnée par stride avec width qui dicte le nombre de blocks par ligne. Le contenu de chaque layer est sérialisé sous forme de chaîne de caractères où chaque caractère représente un certain objet. La champ offset permet plus de précision si besoin.

Dans notre cas, on a :

- x → block
- f → fez
- h → haut de forme
- b → béret
- p → porte chapeau

```

let f chr layer position =
  if chr = 'x' then
    Block.create layer position Rect.{ width = 20; height = 20 } Texture.black |>
    ignore
  else if chr = '@' then

```

```

    Player.create layer position [| Global.get_texture "extra_character_a" |] |>
ignore
    else if chr = 'f' then
        Hat.create position.x position.y layer (Global.get_texture "fez") Fez
    else if chr = 'h' then
        Hat.create position.x position.y layer (Global.get_texture "hdf") Hdf
    else if chr = 'b' then
        Hat.create position.x position.y layer (Global.get_texture "beret") (Beret
position.y)
    else if chr = 'p' then
        Pc.create position.x position.y layer (Global.get_texture "pc")
    else ()

```

Un niveau est donc donné par une liste de layers ce qui permet d’avoir une superposition de calque.

Pour load un niveau, on utilise la fonction suivante :

```

let load f lvl =
    List.iteri
        (fun layer_idx layer ->
            String.iteri
                (fun idx chr ->
                    let x = idx mod layer.width * layer.stride.width in
                    let y = idx / layer.width * layer.stride.height in
                    let position = Vector.add Vector.{ x = float x; y = float y }
(get_offset x y layer chr) in
                    f chr layer_idx position)
                layer.contents)
        lvl.layers;
    let zoom, pos = lvl.camera in
    Global.update (fun g -> { g with camera = { zoom; pos } })

```

La gestion des layers est ainsi naturellement gérée par l’ordre par laquelle on itère sur la variable layer.

Méthodologie de tests

Nous avons plusieurs fonctionnalités à tester :

- Les chapeaux
- La physique
- Les niveaux

Voici notre méthode :

Faisabilité des niveaux

Tout d'abord, il faut nous assurer que nos niveaux sont finissables, donc nous les testons en jouant afin de voir si le niveau se termine.

L'idée est aussi de noter les problèmes rencontrés : de physique, de fonctionnement du chapeau, ou de géométrie du niveau.

Si le problème concerne uniquement le niveau, il est possible de le modifier pour le rendre finissable.

Les niveaux sont ce que nous avons de plus proche d'une suite de tests reproductibles, il s'agit donc de la base des tests et de la découverte des bugs (après tout, si un bug n'est pas déclenché dans les niveaux, est-ce vraiment un bug :) ?)

Chapeaux

Les chapeaux introduisent des comportements complexes au jeu qu'il est nécessaire de tester.

Lorsqu'on voit qu'un chapeau se comporte étrangement, on commence par essayer d'isoler le problème : test sur un niveau plat avec juste la géométrie nécessaire pour le reproduire.

Physique

C'est là que c'est particulièrement amusant : nous avons essayé de marcher dans des murs, lancer des chapeaux dans des murs, sauter et marcher dans des murs, faire des double-sauts.

Certains chapeaux interagissent avec la physique; en particulier le béret. Pour ce dernier, il a fallu faire pas mal d'essais (sauter sur le béret, bloquer le béret, lancer le béret contre un mur, dans le vide, etc) afin d'identifier les problèmes.

Outils techniques

Afin de mener les tests à bien, nous nous sommes dotés de quelques outils techniques.

Déplacement arbitraire du joueur

Pour pouvoir tester plus rapidement, il est utile de pouvoir contrôler les entités (et en particulier le joueur) comme bon nous semble.

A cet effet, un clique n'importe où sur la fenêtre permet de déclencher une téléportation du joueur au niveau du curseur.

Affichages terminal

Notre jeu ne demande pas énormément de debuggage, ainsi, une solution satisfaisante pour le suivi des tests est d'utiliser le terminal pour afficher des informations sur la vitesse d'une entité, sa position, la détection d'une collision, etc...

Utilisation de la couleur

Pour correctement visualiser les collisions, nous avons utilisé au début de notre projet un affichage avec des couleurs différentes selon les collisions en cours.

Répartition du travail

Lucas

NOTE

Comme j'écris du code très vite, j'ai implémenté beaucoup du code du projet; néanmoins chaque fonctionnalité, code et choix technique est le fruit d'un travail de discussion dans le binôme.

Ces parties du jeu me sont attribuables en grande partie:

- Le système d'affichage avec zoom et positionnement de la caméra
- Le système de **Level**, permettant de charger/décharger des niveaux du jeu
- Le système permettant d'attraper/lancer des chapeaux
- Le système d'interpolation du temps pour permettre de ne pas rater les collisions.
- Le système de build reproductibles avec Nix

NOTE

Après beaucoup de discussions en binômes; j'ai tapé le code, mais il est le fruit d'un travail commun.

Sacha

- Le système de collisions
- Le porte chapeau
- Les textures
- Le map-making
- Le code de chargement des textures par continuations